

Méthodologie logicielle

Mamadou Diahame

ESP, Cheikh Anta Diop University, Dakar, Senegal
diahame.mamadou@esp.sn

Module de Méthodologie de développement logicielle

1

Plan

2

- La phase de préparation
- Software development Life Cycle (SDLC)
- Modèle en cascade
- Modèle en V
- Modèle itérative
- Modèle Agile

La phase de préparation

- Etude préalable
- Etude de faisabilité
- Analyse Fonctionnelle
- Etude d'adéquation
- Constitution du dossier de recette

La phase de préparation

➤ Introduction

- Prendre conscience du projet
- Etudier l'objet du projet
- Pour assurer :
 - Mise en oeuvre est pertinente
 - la stratégie de l'entreprise
- Consiste à analyser et étudier la demande du projet
- A la Fin, vous aurez toutes les infos nécessaires pour commencer la réalisation

La phase de préparation

■ Les questions:

- Quels sont les objectifs du projet?
- Quel est le périmètre du projet?
- Quels sont les besoins des utilisateurs?
- Quelles fonctionnalités retenir?

La phase de préparation

■ Etude préalable

- Une première expression de besoins doit être formulé
- La pertinence et la fiabilité de la demande et de son concept doivent être évalués

La phase de préparation

■ Etude préalable

- Plusieurs facteurs à prendre en compte:
 - Les enjeux du projet
 - Les objectifs globaux du projet (Les grandes fonctionnalités)
 - Le périmètre du projet (utilisateurs cibles)
 - Les contraintes associés à chacun de ces éléments
 - Déterminer si le concept est fiable en terme opérationnel et organisationnel

La phase de préparation

■ Etude préalable

- Acteurs: MOA/MOAD
 - Entrant : Besoin
 - Sortant : Note de cadrage (Rapport d'étude préalable)
-
- Ce document représente la véritable "acte" de naissance du projet.
 - Il est validé par le comité de pilotage

La phase de préparation

■ Etude de faisabilité

■ Vise à analyser:

- La faisabilité économique
- La faisabilité organisationnelle
- La faisabilité technique

La phase de préparation

■ Etude de faisabilité

- Procéder à une estimation:
 - Grossière du coût (Humains, matériels, financières)
 - Des délais envisages
 - Des éventuels retour sur investissement (RSI)

La phase de préparation

■ Etude de faisabilité

- Envisager plusieurs scénarios
- Evaluer les risques de chaque scénario
- Etudier la faisabilité de chaque scénario
- Rédiger les documents suivants:
 - Le planning prévisionnel
 - Le budget prévisionnel
 - Le suivi des risques et des actions

La phase de préparation

■ Etude de faisabilité

- Acteurs : MOA/MOAD/MOE
- Entrant : Note de cadrage
- Sortant : Dossier de suivi (Rapport de faisabilité)

La phase de préparation

■ Analyse fonctionnelle

- Une étude plus approfondie des besoins
- Permet de dégager les fonctionnalités
- Mise au point du cahier des charges fonctionnel (CDCF)

■ Ce document permet à la MOA:

- D'exprimer son besoin de manière fonctionnelle
- Clarifier les contraintes imposées à la MOE

La phase de préparation

■ Analyse fonctionnelle

- Acteurs : MOA/ MOAD
- Entrant : Note de cadrage & la Rapport de faisabilité
- Sortant : CDCF

La phase de préparation

■ Etude d'adéquation

- Le CDCF est transmis à de prestataires comme appel d'offres.
- Les réponses sont analysées à l'aide de la grille d'évaluation.
- A la suite de ce choix, un contrat est rédigé entre la MOA et la MOE

La phase de préparation

- **Etude d'adéquation**
 - Acteurs : MOA & MOE
 - Entrant : CDCF
 - Sortant : Contrat de projet

La phase de préparation

- **La constitution du dossier de recette**

- Deux sous-tâches:

- Les spécifications définies par le CDCF
 - La définition des scénarios définis par des spécifications détaillées

La phase de préparation

■ La constitution du dossier de recette

- Acteurs : MOA/MOE
- Entrant : CDCF & Spécification détaillées
- Sortant : Dossier de recette

Software development lifecycle (SDLC)

- est un processus utilisé par l'industrie du logiciel pour:
 - Concevoir,
 - Développer
 - Et tester des logiciels de haute qualité

Software Development Life Cycle (SDLC)

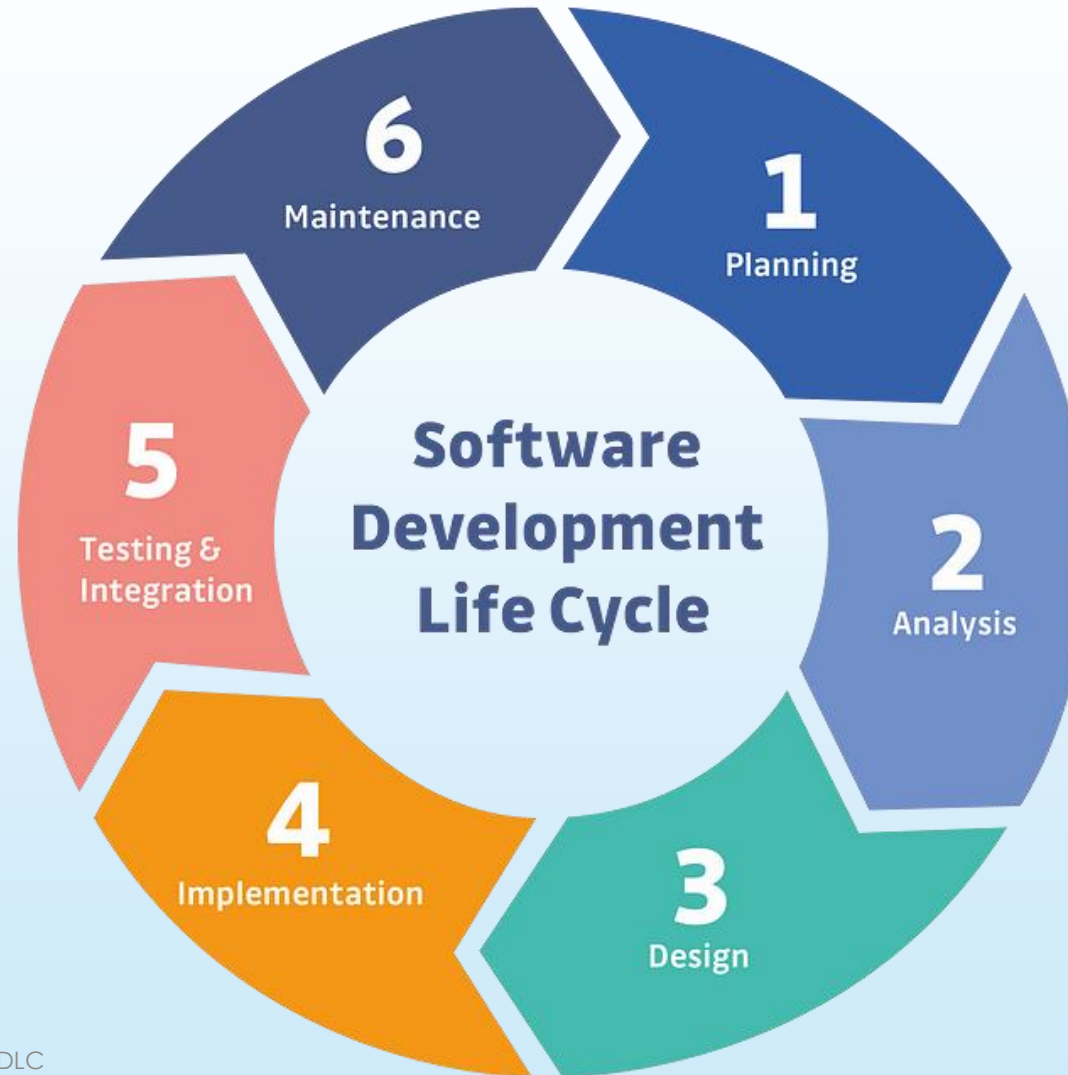
■ Histoire

- Avant les années 1950, l'informatique n'était pas suffisamment élaborée pour nécessiter une approche détaillée telle que le cycle de développement logiciel.
- Au fur et à mesure que la complexité et l'ampleur de la programmation augmentaient, le concept de programmation structurée est apparu.
- Au fil du temps, la programmation structurée a exigé des modèles de développement plus tactiques, ce qui a donné naissance au SDLC.

Software Development Life Cycle (SDLC)

- Il fournit un cadre normalisé qui définit les activités et les produits à livrer.
- Il facilite la planification, l'estimation et l'ordonnancement des projets.
- Il facilite le suivi et le contrôle du projet
- Il améliore la visibilité de tous les aspects du cycle de vie pour toutes les parties prenantes impliquées dans le processus de développement.
- Il augmente la vitesse de développement
- Il améliore les relations avec les clients
- Diminution des risques liés au projet
- Il réduit les dépenses de gestion de projet et le coût global de production.

Software Development Life Cycle (SDLC)



Software Development Life Cycle (SDLC)

- Les 7 étapes du SDLC sont :
 - La planification
 - L'analyse
 - La conception
 - La mise en oeuvre
 - Les tests
 - Le déploiement
 - La maintenance

Software Development Life Cycle (SDLC)

- La planification
 - consiste à identifier les buts et les objectifs du logiciel,
 - les besoins des utilisateurs.
 - implique également: la création d'un plan de projet, qui décrit:
 - les tâches à accomplir
 - les ressources nécessaires à leur réalisation.
- est cruciale car elle jette les bases du reste du projet.
- Si les objectifs ou les exigences ne sont pas bien définis, c'est l'ensemble du projet qui en pâtira.

Software Development Life Cycle (SDLC)

- L'analyse :
 - Les besoins des utilisateurs sont analysés plus en détail.
 - Créer une spécification détaillée du logiciel
 - Déterminer la faisabilité du projet.
- Permet
 - Comprendre si le logiciel est techniquement et économiquement viable
 - D'identifier les ressources
 - Les outils nécessaires
 - Le calendrier de développement.

Software Development Life Cycle (SDLC)

- La conception :
 - Consiste à créer un plan détaillé de la manière dont le logiciel sera développé:
 - Concevoir l'interface utilisateur,
 - l'architecture
 - les composants du logiciel.
 - Il est essentiel d'avoir une conception claire:
 - Définir la structure du logiciel et la manière dont les différentes parties interagiront,
 - Permet également à l'équipe d'anticiper et de résoudre les problèmes techniques avant la phase de mise en œuvre.

Software Development Life Cycle (SDLC)

- La mise en œuvre :
 - le logiciel est effectivement développé
 - l'écriture du code,
 - les tests unitaires et le débogage.
 - C'est à ce stade
 - Que le logiciel commence à prendre forme,
 - Que le code est écrit et assemblé
 - Et que le logiciel commence à fonctionner comme prévu.

Software Development Life Cycle (SDLC)

- Les tests:
 - le logiciel est testé pour s'assurer
 - qu'il répond aux exigences des utilisateurs
 - qu'il ne comporte pas de bugs.
 - Plusieurs types de tests peuvent être effectués à ce stade, tels que
 - les tests unitaires,
 - les tests d'intégration
 - les tests d'acceptation par l'utilisateur.
 - Il est essentiel d'avoir une phase de test complète, car elle permet
 - d'identifier les problèmes et les bugs dès le début du processus
 - d'éviter des erreurs coûteuses.

Software Development Life Cycle (SDLC)

- Le déploiement:
 - Une fois que le logiciel a été testé et jugé stable, il est déployé auprès des utilisateurs.
 - Consiste à installer le logiciel sur l'ordinateur d'un utilisateur
 - Ou à le déployer sur un serveur.
- Il est essentiel de mettre en place une stratégie de déploiement claire, afin de s'assurer que le logiciel atteint les utilisateurs finaux d'une manière fluide et contrôlée.

Software Development Life Cycle (SDLC)

- La maintenance :
 - Le logiciel déployé doit être entretenu afin
 - De corriger les éventuels bugs découverts
 - d'apporter des améliorations.
 - Cette étape est continue et a pour but de maintenir le logiciel à jour, sécurisé et fonctionnel.
- La maintenance du logiciel est importante pour assurer la longévité du logiciel et pour maintenir la satisfaction des utilisateurs.

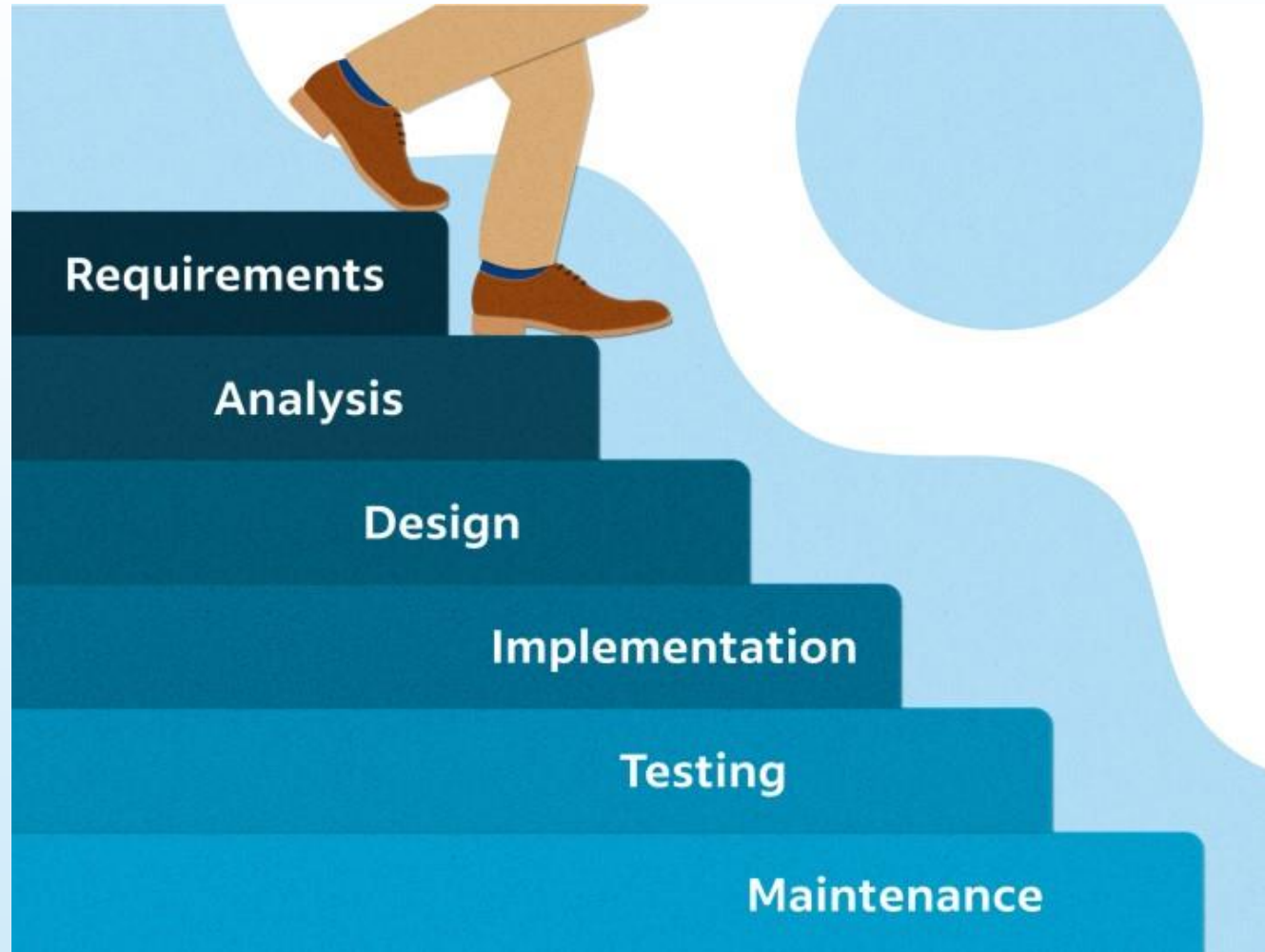
Software Development Life Cycle (SDLC)

- Le modèle de développement logiciel:
 - Cascade
 - V-Shaped
 - Iterative
 - Agile

Modèle en cascade

- parfois appelée cycle de vie séquentiel linéaire ou "waterfall model"
- est une méthode de gestion de projet qui aborde le développement dans un ordre linéaire et séquentiel, ou en cascade.
- Les équipes de développement de projet suivent une progression d'étapes logiques tout au long du cycle de vie du développement logiciel (SDLC).
- Le processus ne passe à la phase suivante du projet que lorsque l'étape précédente a été achevée.

Modèle en cascade



Modèle en cascade

- Cette méthode comporte généralement 5 à 7 phases.
- Varie en fonction du développeur ou du projet.
- Mais les concepts restent généralement les mêmes.
- Les phases de base du modèle en cascade sont:
 - Exigences
 - L'analyse
 - La conception
 - La mise en œuvre
 - Les tests
 - La maintenance

Modèle en cascade

■ Exigences

- Les exigences du client concernant le produit final sont rassemblées au début du projet et compilées dans un document d'exigences
- Ces exigences énumèrent généralement ce que l'application doit faire sans tenir compte de la manière dont elle sera réalisée.
- En recueillant les exigences au début du projet, il n'est pas nécessaire d'impliquer davantage le client et l'équipe peut planifier toutes les autres phases.

Modèle en cascade

- L'analyse
 - On examine les exigences.
 - On élabore une conception pour y répondre.
- Une fois les exigences recueillies, le système est évalué afin de
 - déterminer les outils nécessaires
 - de créer les modèles qui seront utilisés dans l'application finale.
- C'est à ce stade que votre équipe identifie le chemin qu'elle empruntera pour fournir une solution et les spécifications correspondantes.

Modèle en cascade

- La conception
 - La phase de conception peut être divisée en deux sous-phases :
 - **La conception logique** : Des solutions sont imaginées pour répondre aux exigences souhaitées.
 - **La conception physique** : les idées théoriques sont transformées en plans concrets et en spécifications de conception, telles que les services, les couches de données et les langages de programmation.

Modèle en cascade

- La mise en œuvre
 - Le code réel est écrit pour répondre aux spécifications, aux modèles et aux exigences qui ont été définis dans les phases précédentes.
 - Des tests peuvent avoir lieu et si des changements importants sont nécessaires au cours de cette phase, il peut être nécessaire de revenir à la phase de conception.

Modèle en cascade

- Les tests:
 - Une fois le code écrit, les bêta-testeurs, les responsables de l'assurance qualité et tous les autres testeurs concernés explorent systématiquement l'application.
 - Cette phase, également connue sous le nom de phase de vérification, nécessite généralement une répétition de la phase de codage afin de corriger les bugs découverts.
 - Parfois, cette phase implique également le client, ce qui lui permet de tester le produit pour s'assurer que toutes les exigences ont été correctement satisfaites.

Modèle en cascade

- La Maintenance
 - l'application est prête à être diffusée.
 - Cette dernière phase consiste souvent à assurer un soutien continu en maintenant l'application à jour et fonctionnelle, ainsi qu'en corrigeant les bugs ou les erreurs lorsqu'ils sont détectés.

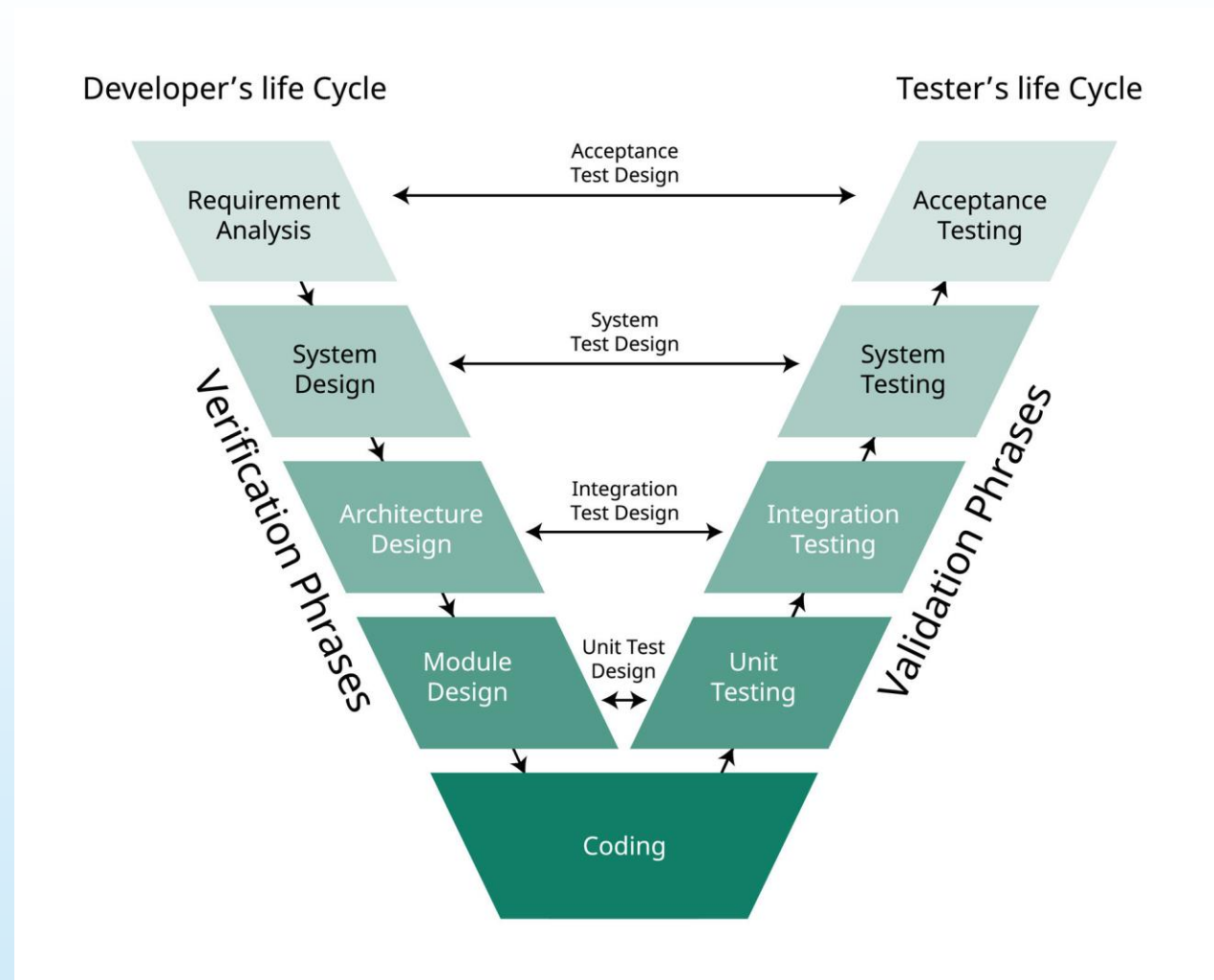
Modèle en cascade

- La Maintenance
 - l'application est prête à être diffusée.
 - Cette dernière phase consiste souvent à assurer un soutien continu en maintenant l'application à jour et fonctionnelle, ainsi qu'en corrigeant les bugs ou les erreurs lorsqu'ils sont détectés.

Modèle En V

- Le modèle en V est utile lorsque l'on travaille avec des équipes plus importantes et que la communication et la coordination entre les développeurs et les testeurs deviennent difficiles.
- En définissant clairement les exigences de test pour chaque étape du processus de développement, le modèle en V peut garantir que tous les membres de l'équipe travaillent dans le sens d'une compréhension commune des buts et objectifs du projet.
- Il peut être subdivisé en deux parties:
 - La phase de vérification
 - La phase de validation

Modèle En V



Modèle En V

- La phase de vérification consiste à évaluer le processus de développement du produit afin de s'assurer que l'équipe répond aux exigences spécifiées.
- La phase de vérification comprend plusieurs étapes :
 - l'analyse des besoins de l'entreprise,
 - l'analyse du système,
 - la conception de l'architecture logicielle,
 - la conception des modules et
 - le codage.

Modèle En V

- l'analyse des besoins de l'entreprise:
 - l'équipe comprend les exigences du produit telles qu'elles ont été définies par le client
- l'analyse du système :
 - Les ingénieurs système analysent et interprètent les exigences commerciales du système proposé
 - en étudiant le document relatif aux exigences de l'utilisateur.
- la conception de l'architecture logicielle :
 - sélectionne l'architecture logicielle sur la base
 - de la liste des modules,
 - des fonctionnalités succinctes de chaque module,
 - des relations d'interface,
 - des dépendances,
 - des tables de base de données,
 - des diagrammes d'architecture,
 - des détails technologiques, etc.
 - Le modèle de test d'intégration est élaboré au cours de cette phase

Modèle En V

- la conception des modules :
 - l'équipe de développement
 - décompose le système en petits modules
 - spécifie la conception détaillée de chaque module,
 - Ce que nous appelons la conception de bas niveau.
- Le codage
 - choisit un langage de programmation approprié en fonction de la conception et des exigences du produit.
 - Il existe bien sûr des lignes directrices et des normes pour le codage
 - Le code sera soumis à de nombreuses révisions pour en vérifier les performances.

Modèle En V

- La conception des modules :
 - l'équipe de développement
 - décompose le système en petits modules
 - spécifie la conception détaillée de chaque module,
 - Ce que nous appelons la conception de bas niveau.

Modèle En V

- La phase de validation fait appel à des méthodes d'analyse dynamique et à des tests pour s'assurer que le produit logiciel répond aux exigences et aux attentes du client.
- Cette phase comprend plusieurs étapes, notamment:
 - les tests unitaires,
 - les tests d'intégration,
 - les tests du système
 - les tests d'acceptation.

Modèle En V

- la phase de tests unitaires:
 - l'équipe élabore et exécute des plans de tests unitaires afin d'identifier les erreurs au niveau du code ou de l'unité.
 - Ces tests portent sur les plus petites entités, telles que les modules de programme, afin de s'assurer qu'elles fonctionnent correctement lorsqu'elles sont isolées du reste du code.

Modèle En V

- La phase de test d'intégration:
 - implique l'exécution de plans de test d'intégration développés au cours de l'étape de conception architecturale afin de vérifier que les groupes créés et testés indépendamment peuvent coexister et communiquer les uns avec les autres.

Modèle En V

- L'étape de test du système:
 - consiste à exécuter les plans de test du système élaborés au cours de l'étape de conception du système, qui sont composés par l'équipe commerciale du client. Les tests du système permettent de s'assurer que l'équipe répond aux attentes du développeur de l'application.

Modèle En V

- L'étape des tests d'acceptation:
 - est liée à la partie du modèle en V consacrée à l'analyse des besoins de l'entreprise et consiste à tester le produit logiciel dans l'environnement de l'utilisateur afin d'identifier les problèmes de compatibilité avec les différents systèmes disponibles dans l'environnement de l'utilisateur. Les tests d'acceptation permettent également d'identifier les problèmes non fonctionnels tels que les défauts de charge et de performance dans l'environnement réel de l'utilisateur.

Modèle En V

- Principes du modèle en V
 - Le modèle en V souligne l'importance des tests et de l'assurance qualité tout au long du processus de développement. Voici quelques-uns des principes clés du modèle V.

Modèle En V

- INTÉGRER LES TESTS TOUT AU LONG DU DÉVELOPPEMENT
 - Le test n'est pas seulement une activité qui se déroule à la fin du processus de développement. Au contraire, les tests sont intégrés à chaque étape du cycle de développement, de la définition des besoins au déploiement.
- PLANIFIER LES TESTS PARALLÈLEMENT AU DÉVELOPPEMENT
 - À chaque étape du processus de développement correspond une phase de test. Les activités de test sont planifiées parallèlement aux activités de développement afin que les ressources nécessaires soient disponibles pour soutenir les tests.
- PRÉVENIR LES DÉFAUTS
 - Le modèle en V met l'accent sur l'importance de prévenir les défauts plutôt que de simplement les identifier et les corriger une fois qu'ils ont été découverts.

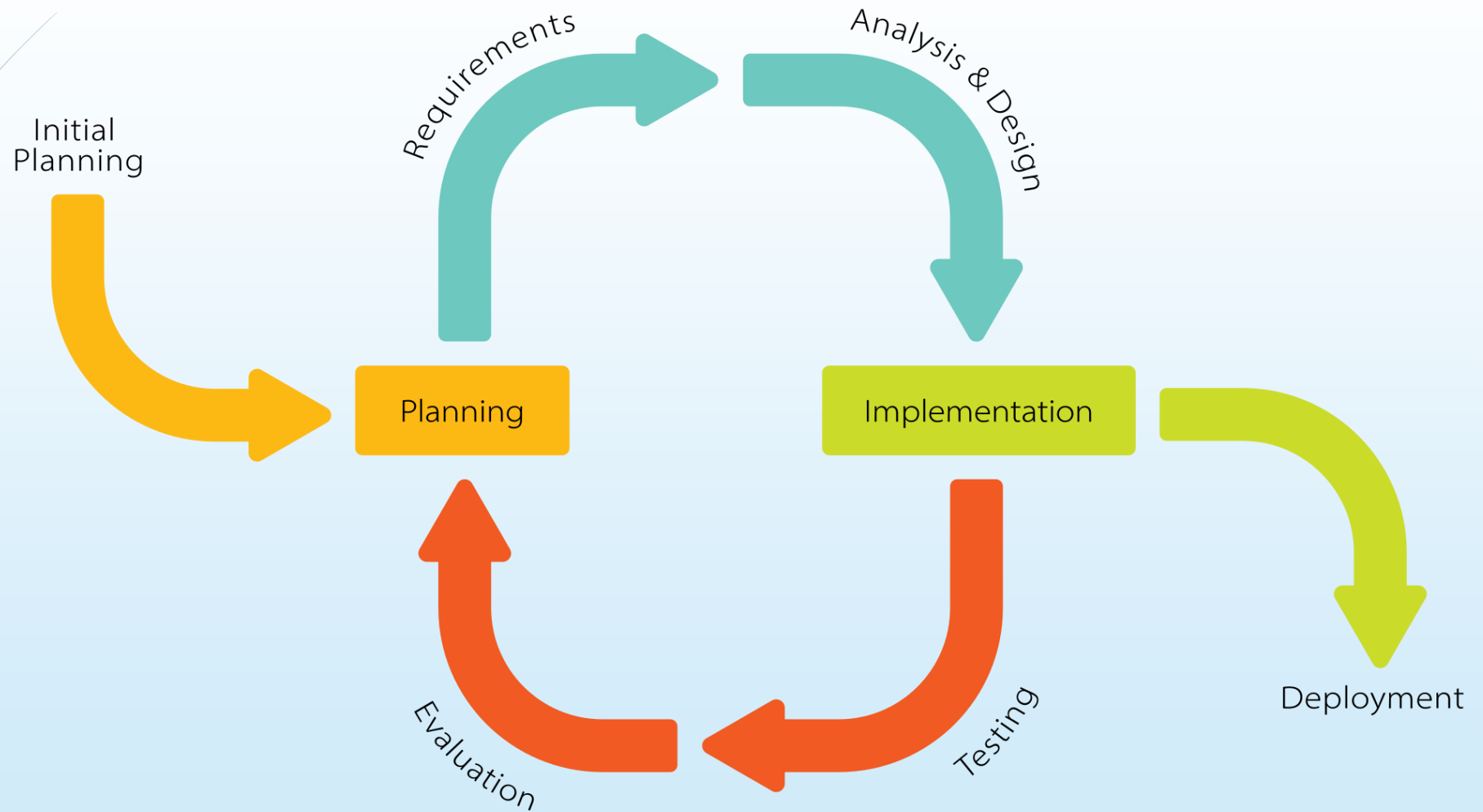
Modèle En V

- ▶ ÉLABORER DES EXIGENCES CLAIRES ET CONCISES
 - ▶ Le modèle en V met l'accent sur des exigences claires et concises. Sans une compréhension claire de ce que le logiciel est censé faire, il est impossible de développer des tests efficaces ou de construire un logiciel de haute qualité.
- ▶ COMBINER LE DÉVELOPPEMENT ET LES TESTS
 - ▶ Dans le modèle en V, le développement et les tests ne sont pas des activités séparées. Au contraire, elles sont étroitement intégrées et la collaboration entre les développeurs et les testeurs est essentielle pour garantir que le logiciel répond aux normes de qualité requises.

Modèle Itératif

- Également connue sous le nom de méthode de développement incrémental de logiciels, cette méthodologie s'applique aux produits qui sont développés sur une base incrémentale.
- Le processus commence par un produit logiciel à moitié terminé, puis les développeurs continuent d'intégrer de nouvelles caractéristiques et fonctionnalités au logiciel jusqu'à ce que celui-ci soit complètement achevé.

Modèle Itératif



Modèle Agile

- Décomposer vos projets en une suite de petits objectifs atteignables.
- Vous travaillez en de petits cycles courts que l'on appelle sprints ou itérations qui durent généralement entre 1 semaine et 1 mois.
- Il existe plusieurs façons d'appliquer la méthodologie Agile parmi lesquels:
 - La méthode Kanban
 - Scrum
 - Extreme Programming (XP)

Modèle Agile

- Les avantages:
 - Flexibilité:
 - vous travaillez en flux tendu.
 - Vous vous occupez des tâches qui sont importantes à un instant T.
 - Vous êtes donc capable de vous adapter rapidement à votre environnement.
 - Feedback:
 - À la fin de chaque sprint, vous livrez votre travail et récupérez les feedbacks de vos clients ou de vos utilisateurs puis vous les incorporez dans le sprint suivant ce qui permet de coller au plus près à leurs besoins.
 - Compétitivité:
 - vous livrez régulièrement de nouvelles fonctionnalités, de nouveaux produits et de nouvelles améliorations ce qui vous rend compétitif et attractif auprès de vos clients et utilisateurs.
 - Qualité:
 - La livraison fréquente de projets, les tests et l'intégration permanente de feedbacks permettent de développer un produit final de qualité.
 - Pendant tout le cycle de production, le produit est testé et confronté à la réalité.

Modèle Agile

- Les inconvénients:
 - Manque de documentation:
 - Étant donné que le scope des projets change au gré des besoins et des feedbacks, il est difficile de tenir une documentation à jour.
 - La documentation est généralement négligée
 - Gestion des demandes:
 - Intégrer continuellement les demandes des clients et utilisateurs.
 - gérer un plus grand volume de demandes entrantes ce qui peut compliquer la gestion de projet
 - Manque de prévisibilité:
 - Vous ne savez pas toujours à quoi ressemblera le résultat final.
 - Difficile donc de prévoir le coût, le temps et les ressources nécessaires.

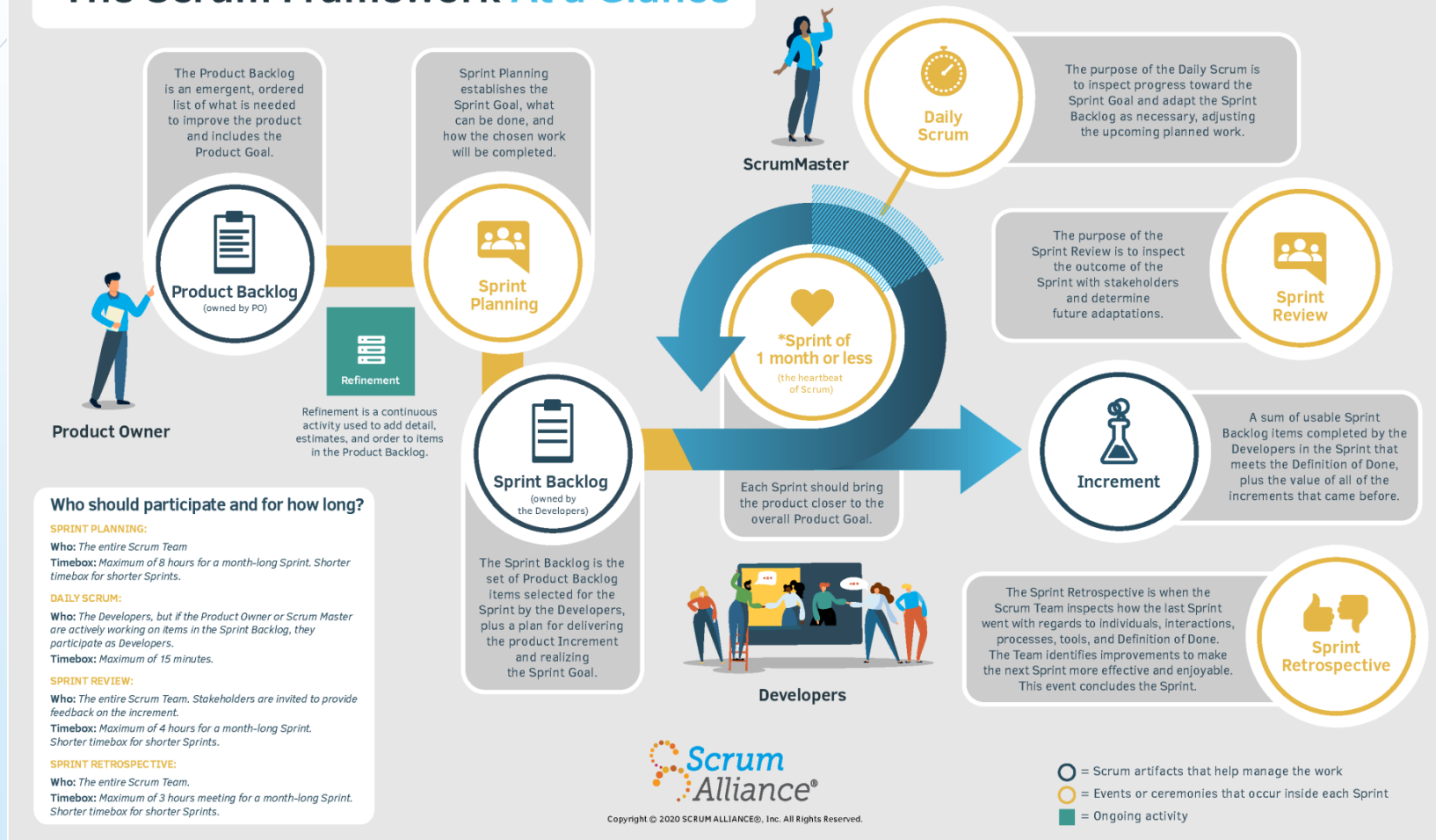
Le framework Scrum

- Scrum est un framework léger qui aide les personnes, les équipes et les organisations à générer de la valeur grâce à des solutions adaptatives pour des problèmes complexes".

Le framework Scrum

The Scrum Framework At a Glance

Version 5.0



Le framework Scrum

■ Les rôles

- Le **Product Owner** qui réalise la vision du projet, c'est lui qui est chargé de maintenir le backlog à jour (le backlog est l'inventaire des tâches à réaliser).
- Le **Scrum Master** qui est garant de la méthodologie Scrum. Il n'a pas le rôle de chef de projet. Il est chargé de promouvoir la méthode Scrum et de s'assurer qu'elle est bien comprise et utilisée.
- **L'équipe qui réalise le projet.** Elle peut inclure différents types de personnes : des développeurs, graphistes, ingénieurs...

Le framework Scrum

■ Les artifacts:

- **Product Backlog** - Liste émergente et ordonnée de ce qui est nécessaire pour améliorer le produit et qui inclut l'objectif du produit.
- **Sprint Backlog** - L'ensemble des éléments du carnet de commandes sélectionnés pour le sprint par les développeurs (membres de l'équipe), ainsi qu'un plan de livraison de l'incrément et de réalisation de l'objectif du sprint.
- **Increment** - Somme des éléments utilisables du backlog du sprint complétés par les développeurs au cours du sprint et répondant à la définition de "done", plus la valeur de tous les incréments qui ont précédé. Chaque incrément est une version reconnaissable, visiblement améliorée et opérationnelle du produit.

Le framework Scrum

■ Les artifacts:

- **Definition of Done** - Lorsque l'incrément est livré, il doit répondre à une compréhension commune de ce que signifie "fait". La définition de "done" garantit que la norme de qualité est respectée. Cette définition peut varier d'une organisation à l'autre et d'une équipe à l'autre.
- **Sprint Goal** - Un objectif spécifique et singulier pour le backlog du sprint. Cet objectif permet à chacun de se concentrer sur l'essentiel de ce qui doit être fait et pourquoi.
- **Product Goal** - Pour planifier le travail à effectuer à chaque sprint, les équipes doivent avoir une idée de l'objectif global de leur produit. Chaque équipe peut avoir plusieurs objectifs de produit au cours de sa vie, mais un seul à la fois.

Le framework Scrum

- Les événements:
 - **The Sprint:**
 - Le rythme cardiaque de scrum.
 - Chaque sprint doit permettre de rapprocher le produit de son objectif et durer un mois ou moins.
 - **Sprint Planning:**
 - L'ensemble de l'équipe de scrum définit l'objectif du sprint, ce qui peut être fait et comment le travail choisi sera réalisé.
 - La planification doit se faire dans un délai maximum de 8 heures pour un sprint d'un mois, et dans un délai plus court pour les sprints plus courts.

Le framework Scrum

■ Les événements:

■ **Daily Scrum:**

- Les développeurs (les membres de l'équipe qui exécutent le travail) vérifient les progrès accomplis dans la réalisation de l'objectif du sprint et adaptent le backlog du sprint si nécessaire, en ajustant le travail planifié à venir.
- Une mêlée quotidienne devrait durer 15 minutes chaque jour.

■ **Sprint Review:**

- L'ensemble de l'équipe scrum examine le résultat du sprint avec les parties prenantes et détermine les adaptations futures.
- Les parties prenantes sont invitées à donner leur avis sur l'incrément.

■ **Sprint Retrospective:**

- L'équipe scrum examine comment s'est déroulé le dernier sprint en ce qui concerne les individus, les interactions, les processus, les outils et la définition de ce qui a été fait.
- L'équipe identifie les améliorations à apporter pour rendre le prochain sprint plus efficace et plus agréable. C'est la conclusion du sprint.

Le framework Scrum

- Les piliers:
 - La transparence
 - Pour prendre des décisions, il faut avoir une visibilité sur le processus et sur l'état actuel du produit.
 - Pour que chacun comprenne ce qu'il voit, les participants doivent partager un même langage.
 - Les Sprints reviews assure la transparence.
 - Les Revues fréquentes du sprint donnent aux membres de l'équipe et aux parties prenantes une vision claire de l'état d'avancement du projet

Le framework Scrum

- Les piliers:
 - L'inspection
 - Pour éviter tout écart par rapport au processus ou au produit final souhaité, il est nécessaire d'inspecter ce qui est créé, et comment, à intervalles réguliers.
 - L'inspection doit avoir lieu au moment où le travail est effectué, mais ne doit pas entraver ce travail.
 - Sprint reviews et retrospectives offrent des opportunités d'inspection. En effet, Les équipes Scrum contrôlent leur travail et leur processus à la fin de chaque itération lors des revues de sprint et des rétrospectives de sprint.

Le framework Scrum

- Les piliers:
 - L'adaptation
 - L'adaptation signifie que lorsque des écarts se produisent, le processus ou le produit doit être ajusté dès que possible.
 - Les équipes Scrum peuvent adapter le produit à la fin de chaque sprint. Scrum permet des ajustements à la fin de chaque itération.
 - Les équipes Scrum peuvent adapter le produit à la fin de chaque sprint.
 - Scrum permet des ajustements à la fin de chaque itération.